

Projekt DBI: Datenbank & REST-API

Ziel ist es, ein vollständiges datenbankgestütztes Backend zu entwickeln. Ihr wählt selbst ein Thema aus dem Alltag (z.B. Bibliothek, Sportverein, Rezeptverwaltung, Kinoprogramm, ...) und baut dafür ein durchgängiges System: vom Datenbankmodell bis zur lauffähigen REST-API. Frontend wird im POS1-Unterricht gebaut.

! Wichtig

Keep it simple! Eine sauber umgesetzte kleine Domäne ist besser als ein halbfertiges, zu groß gedachtes Projekt. Bewertet wird die Qualität der Umsetzung, nicht die Anzahl der Tabellen.

Organisatorisches

- Idealerweise 2er Teams
- Arbeit teils im Unterricht, teils zuhause
- Themenwahl muss vorab vom Lehrer freigegeben werden

Benotung

- Erfüllung der Grundanforderungen => max. 3er möglich
- Einflussfaktoren für bessere Noten
 - Vollständigkeit und Korrektheit der Datenbankdokumentation (ERM, RM, Normalformen)
 - Qualität und Sinnhaftigkeit der API-Endpunkte
 - Sicherheit (Validierung, SQL-Injection-Schutz)
 - Demo in Hälfte der Projektzeit
 - Komplexität und Kreativität der gewählten Domäne
 - Optionale Features umgesetzt?
 - Präsentation
 - Gesamteindruck

Technische Grundanforderungen

Datenbankmodell

- **Mindestens 5 Tabellen** mit sinnvollem fachlichem Zusammenhang ✓
- Alle drei Kardinalitäten müssen vorkommen und im ERM beschriftet sein:
 - **1:1** (z.B. Person ↔ Ausweis) ✓
 - **1:n** (z.B. Autor → Bücher) ✓
 - **m:n** (z.B. Studenten ↔ Kurse, umgesetzt als Zwischentabelle) ✓
- **ERM** als Diagramm (DrawIO, PlantUML o.ä.) mit Entitäten, Attributen und Kardinalitäten ✓
- **Relationales Modell (RM)** vollständig aus dem ERM abgeleitet: ✓
 - Alle Tabellen mit Spalten, Primärschlüsseln (unterstrichen) und Fremdschlüsseln (kursiv) ✓
 - m:n-Beziehungen als eigene Zwischentabelle ✓

① Hinweis

Eine kurze Erklärung pro Tabelle, warum sie die jeweilige Normalform erfüllt, muss gemacht werden.

REST-API (FastAPI + SQLite)

Stack:

- Python + **FastAPI** + **uvicorn**
- SQLite als Datenbank (muss nicht SQLite sein)
- **Pydantic**-Modelle für alle Request- und Response-Schemas

Endpunkte – Pflicht:

Alle vier CRUD-Operationen müssen für mindestens eine Hauptressource vollständig implementiert sein:

Methode	Pfad	Bedeutung
GET	<code>/ressource</code>	Alle Einträge auflisten (optional: Filterparameter)
GET	<code>/ressource/{id}</code>	Einzelnen Eintrag abrufen
POST	<code>/ressource</code>	Neuen Eintrag anlegen
PUT	<code>/ressource/{id}</code>	Eintrag vollständig aktualisieren
DELETE	<code>/ressource/{id}</code>	Eintrag löschen

! Wichtig

Alle fünf Endpunkte (GET-Liste, GET-Einzel, POST, PUT, DELETE) müssen tatsächlich funktionieren und korrekte Statuscodes zurückgeben. Es darf **keinen generischen Endpunkt** geben, der beliebige SQL-Abfragen entgegennimmt und ausführt. Jeder Endpunkt hat eine klar definierte, fachliche Bedeutung.

JOINS und Aggregation – Pflicht:

Die API muss zeigen, dass die Datenbank sinnvoll abgefragt wird:

- **Mindestens ein Endpunkt** muss Daten aus **zwei oder mehr Tabellen per JOIN** zusammenführen (z.B. `GET /orders` gibt Bestellungen inkl. Kundenname zurück)
- **Mindestens ein Aggregationsendpunkt** muss `GROUP BY` mit einer Aggregatfunktion (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`) verwenden (z.B. `GET /stats/orders-per-customer`)

```
-- Beispiel JOIN
SELECT orders.id, customers.name, orders.total
FROM orders
JOIN customers ON orders.customer_id = customers.id

-- Beispiel Aggregation
SELECT customers.name, COUNT(orders.id) AS order_count
FROM customers
LEFT JOIN orders ON customers.id = orders.customer_id
GROUP BY customers.id
```

JSON-Responses – Pflicht:

Alle Antworten der API müssen JSON sein – sowohl Erfolg als auch Fehler. Responses müssen inhaltlich sinnvoll sein und den Zustand klar kommunizieren.

Beispiele für korrekte Responses:

```
// POST /books → 201 Created
{ "title": "Der Prozess", "author_id": 2, "year": 1925 }
```

```
// DELETE /books/5 → 200 OK
{ "message": "Buch erfolgreich gelöscht", "id": 5 }

// GET /books/99 → 404 Not Found
{ "detail": "Buch mit ID 99 nicht gefunden" }

// POST /books mit fehlendem Pflichtfeld → 422 Unprocessable Entity
{ "detail": [{ "loc": ["body", "title"], "msg": "field required", "type": "value_error.missing" }] }

// POST /books mit ungültigem Wert → 400 Bad Request
{ "detail": "Das Erscheinungsjahr darf nicht in der Zukunft liegen" }
```

! Wichtig

Fehlermeldungen müssen **erklären, was falsch ist** – nicht nur einen Statuscode zurückgeben. Ein leerer Body oder eine generische Meldung wie `"error": "bad request"` ist nicht ausreichend.

HTTP-Statuscodes – Pflicht:

Jeder Endpunkt muss den semantisch korrekten HTTP-Statuscode zurückgeben:

Situation	Code
GET / PUT erfolgreich	200 OK
POST erfolgreich (Ressource erstellt)	201 Created
DELETE erfolgreich	200 OK
Ressource nicht gefunden	404 Not Found
Ungültige Eingabe (Typ, Format, Wertebereich)	422 Unprocessable Entity (Pydantic automatisch)
Fachlicher Validierungsfehler (z.B. Jahr in Zukunft)	400 Bad Request
Konflikt (z.B. eindeutiger Wert bereits vorhanden)	409 Conflict

① Hinweis

FastAPI gibt bei Pydantic-Fehlern automatisch 422 zurück. Eigene fachliche Prüfungen (z.B. Altersgrenze, Datum in Vergangenheit) müssen manuell mit `HTTPException` und dem richtigen Code geworfen werden.

Validierung – Pflicht:

- Alle Eingabefelder müssen über Pydantic-Modelle validiert werden (Typ, Pflichtfelder, Wertebereiche)
- Fachliche Regeln (z.B. Alter > 0, Datum nicht in der Zukunft) werden zusätzlich im Endpunkt geprüft
- Bei nicht gefundener Ressource: 404 Not Found

SQL-Injection-Schutz – Pflicht:

- Alle Datenbankzugriffe müssen **parametrisierte Abfragen** verwenden
- String-Konkatenation oder f-Strings für SQL-Befehle sind **nicht erlaubt**

```
# ☒ Richtig - parametrisierte Abfrage
cursor.execute("SELECT * FROM books WHERE author_id = ?", (author_id,))

# ☐ Falsch - anfällig für SQL-Injection
cursor.execute(f"SELECT * FROM books WHERE author_id = {author_id}")
```

KI-Kennzeichnung:

Code der mithilfe von KI generiert wurde muss folgendermaßen gekennzeichnet werden:

```
# KI-Anfang
# KI: ChatGPT
# prompt: Wie schreibe ich einen FastAPI-Endpoint mit Pydantic?
@app.get("/books")
...
# KI-Ende
```

Rollen – Pflicht:

Die API muss zwei Rollen mit unterschiedlichen Rechten unterstützen z.B.:

Beispiel

Rolle	Erlaubte Operationen
admin	GET, POST, PUT, DELETE
user	nur GET

Authentifizierung - Optional:

Umsetzung über **API-Key im Header** (X-API-Key) oder **JWT-Token**:

```
# Beispiel API-Key
GET /books HTTP/1.1
X-API-Key: mein-geheimer-schlüssel
```

- Endpunkte müssen bei fehlendem oder ungültigem Token **401 Unauthorized** zurückgeben
- Die Keys/Tokens dürfen hardcodiert oder in einer `.env`-Datei gespeichert sein

Hinweis

Empfehlung: API-Key-Authentifizierung ist einfacher als JWT und reicht für dieses Projekt aus. Die Bibliothek `python-dotenv` hilft beim Laden von `.env`-Dateien.

Logging – Pflicht:

Jede API muss alle eingehenden Requests und Fehler loggen. Verwende Pythons eingebautes `logging`-Modul:

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(message)s",
    handlers=[
```

```

        logging.FileHandler("api.log"),
        logging.StreamHandler()
    ]
)

logger = logging.getLogger(__name__)

# In jedem Endpunkt:
logger.info("GET /books aufgerufen")
logger.error("Fehler beim Abrufen von Buch ID %s: %s", book_id, str(e))

```

- **INFO** für normale Requests (Methode, Pfad, Statuscode)
- **ERROR** für unerwartete Fehler
- Logs müssen sowohl in die Konsole als auch in eine Datei ([api.log](#)) geschrieben werden

GIT - Pflicht:

- Jedes Gruppenmitglied committed regelmäßig
- Die Commits sind sinnvoll benannt

Externe Bibliotheken

Weitere Python-Bibliotheken dürfen verwendet werden (z.B. [python-jose](#) für JWT, [passlib](#) für Passwort-Hashing), solange sie in der [requirements.txt](#) eingetragen sind. Die Verwendung einer Bibliothek allein wertet das Projekt nicht auf.

Optionale Erweiterungen (für bessere Noten)

Aufgabe

Erweiterte Filterung: Ergänze GET-Endpunkte um sinnvolle Query-Parameter (z.B. Suche nach Name, Filtern nach Datum, Sortierung).

Aufgabe

Pagination: Implementiere Seitennummerierung für Listenendpunkte über [limit](#) und [offset](#).

Aufgabe

Cloud-Datenbank: Verbinde die API mit einer Datenbank in der Cloud (z.B. PlanetScale, Railway, Supabase) statt einer lokalen SQLite-Datei.

Aufgabe

Erweiterte Aggregation und Statistik: Ergänze GET-Endpunkte um sinnvolle Aggregation und Statistik funktionalitäten.

Ablauf

1. Team bilden und Thema wählen
2. Planungsphase (→ Freigabe durch Lehrer)
3. Umsetzungsphase
4. Präsentation und Abgabe

2. Planungsphase

Jedes Team legt folgendes schriftlich fest (kurzes Dokument, vor Umsetzungsstart abzugeben):

- **Projektbeschreibung:** Welche Domäne? Was soll das System verwalten?

- **Erster ERM-Entwurf**
- **Erster RM-Entwurf**
- **1NF bis 3NF**
- **Einteilung:** Must-Have vs. Nice-To-Have Features
- **Aufgabenverteilung:** Wer macht was?

Nach Freigabe durch den Lehrer startet die Umsetzungsphase.

3. Umsetzungsphase

Arbeitet schrittweise: erst Datenbankmodell fertigstellen, dann API aufbauen. Haltet Probleme und Lösungen im Projekttagbuch fest.

4. Präsentation und Abgabe

Die Präsentation zeigt:

1. Kurze Vorstellung der Domäne
2. ERM und RM erklären
3. Live-Demo der API (z.B. über Swagger UI unter [/docs](#))
4. Reflexion: Was lief gut, was würdet ihr anders machen?

Abgabeform:

- Dokumentation als PDF
 - Projektbeschreibung
 - ERM (Diagramm)
 - RM (Tabellenübersicht mit Schlüsseln)
 - Normalisierungsnachweis (1NF, 2NF, 3NF)
 - Projekttagbuch (wer hat wann was gemacht)
 - Bedienungsanleitung (wie startet man die API, Beispiel-Requests)
- Projektordner als ZIP

Ordnerstruktur:

```
ProjektName/  
├── requirements.txt  
├── init_db.py  
├── src/  
│   ├── main.py  
│   ├── database.py  
│   ├── models.py  
│   └── routers/  
│       ├── ressource1.py  
│       └── ressource2.py  
├── doc/  
│   ├── DBI_Dokumentation_ProjektName.pdf  
│   └── erm.drawio
```

! Beachte

Die API muss durch `uvicorn src.main:app` startbar sein. Alle Abhängigkeiten müssen in `requirements.txt` stehen.